

Databricks 14 Days AI Challenge

Day 3: PySpark Transformations Deep Dive (11/01/26)

1. PySpark vs Pandas comparisons

Aspect	Pandas	PySpark
Execution model	Runs on a single machine	Runs on a distributed cluster
Data handling	Loads entire data into memory	Data is split into partitions across nodes
Scale	Limited by RAM of one system	Designed for TB–PB scale
Performance	Very fast for small datasets	Optimized for large datasets
Execution style	Eager execution	Lazy evaluation with query optimization
Failure tolerance	Low (process crash = failure)	High (retries, fault tolerant)
Best for	Quick analysis and prototyping	Production-grade data pipelines
Joins & aggregations	Slows down as data grows	Handles heavy joins and aggregations efficiently
Typical environment	Local machine / small server	Databricks, EMR, Dataproc, clusters

Pandas is single-node and eager, while PySpark is distributed and lazy, making PySpark suitable for large-scale data processing.

2. Joins (inner, left, right, outer)

MYSQL logic	Pyspark logic
SELECT * FROM orders o INNER JOIN customers c ON o.customer_id = c.customer_id;	orders.join(customers, "customer_id", "inner").show()
SELECT * FROM orders o LEFT JOIN customers c ON o.customer_id = c.customer_id;	orders.join(customers, "customer_id", "left").show()
SELECT * FROM orders o RIGHT JOIN customers c ON o.customer_id = c.customer_id;	orders.join(customers, "customer_id", "right").show()
SELECT * FROM orders o FULL OUTER JOIN customers c ON o.customer_id = c.customer_id;	orders.join(customers, "customer_id", "outer").show()

Assume: orders(order_id, customer_id, amount)
customers(customer_id, name, city)

Broadcast join

In Spark, **joins are expensive** because data is distributed.

Unlike MySQL (single machine), Spark data is spread across many machines.

Normal Spark Join (Shuffle Join)

Situation

- orders table → **500 GB**
- customers table → **5 GB**
- Both are distributed across executors

What Spark must do (by default)

1. Spark **repartitions both tables** on customer_id
2. Data is **shuffled across the network**
3. Matching partitions are joined

This means:

- heavy network I/O
- disk spills
- slow jobs
- executor pressure

Mental picture

“Everyone sends their data to everyone else until matching keys meet.” That’s expensive.

Broadcast Join — the idea

Key observation

One table is **small**. So instead of moving **both** tables... 👉 **Move the small table to all executors**

Broadcast Join (what actually happens)

Assume:

- orders = 500 GB (big)
- customers = 5 MB (small)

Spark’s strategy

1. Driver collects the **small table** and sends a **copy of it to every executor**
2. Each executor:
 - keeps customers in memory
 - joins it locally with its partition of orders

Mental picture

“Give every worker a copy of the phonebook instead of asking them to fetch names over the network.”

No shuffle. Huge speedup.

Sample data

orders (BIG)

order_id	customer_id	amount
1	101	500
2	102	800
3	103	300

customers (SMALL)

customer_id	name
101	Rahul
102	Anita

MYSQL mindset:

```
SELECT *
```

```
FROM orders o
```

```
JOIN customers c
```

```
ON o.customer_id = c.customer_id;
```

PySpark – normal join (Shuffle join)

```
orders.join(customers, "customer_id", "inner").show()
```

PySpark - Broadcast Join

```
from pyspark.sql.functions import broadcast
```

```
orders.join(broadcast(customers), "customer_id", "inner").show()
```

3. Window functions (running totals, rankings)

Window functions let you: compute values **without collapsing rows**.

Example Data:

```
from pyspark.sql.window import Window
```

```
from pyspark.sql.functions import sum, rank
```

Running totals:

Running total (per customer)

```
python

window_spec = Window.partitionBy("customer_id") \
                    .orderBy("order_date") \
                    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

df.withColumn("running_total",
              sum("amount").over(window_spec)).show()
```

Key idea:

Grouping + ordering + no row loss



We will understand this by an example: For each customer, show every order **and** keep a running total of how much they have spent so far, ordered by date.

customer_id	order_date	amount
101	2024-01-01	100
101	2024-01-05	200
101	2024-01-10	150
102	2024-01-03	300
102	2024-01-08	100

Using window function

```
sql

SELECT
  customer_id,
  order_date,
  amount,
  SUM(amount) OVER (
    PARTITION BY customer_id
    ORDER BY order_date
  ) AS running_total
FROM orders;
```

but if you want to do it in PySpark, you have to follow the steps mentioned above the table.

First define the window by importing -> write the function.

In MYSQL, we don't have to write rows between implicitly but in PySpark we have to write it,

`rowsBetween(Window.unboundedPreceding, Window.currentRow)`

that means, start from the first row of the partition and end at the current row.

Ranking:

Ranking (top customers by spend)

```
python

rank_window = Window.orderBy(df.amount.desc())

df.withColumn("rank", rank().over(rank_window)).show()
```

Use cases:

- leaderboards
- top-N reports
- session analysis

4. User-Defined Functions (UDFs)

UDFs let you write **custom logic**, but this is where people mess up.

Simple UDF example

```
python

from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def price_category(price):
    return "High" if price > 5000 else "Low"

price_udf = udf(price_category, StringType())

df.withColumn("price_segment", price_udf(df.price)).show()
```